Prueba Técnica Croper – Respuestas teóricas

Desarrollador Full Stack · github.com/crhistian27/prueba-tecnica-croper

3. Arquitectura de microservicios

- **Empezar con un monolito modular** (como el backend NestJS) y separar servicios solo cuando haga falta.
- **Servicios:** Catálogo, Búsqueda, Pedidos, Inventario, Pagos, Logística y Notificaciones, cada uno con su propia base de datos.
- **Entrada:** un API Gateway que valida el JWT; el login lo maneja un proveedor de identidad (OIDC).
- **Comunicación:** REST para consultar y eventos (RabbitMQ) para reaccionar, por ejemplo “pedido creado” o “pago confirmado”.
- **Compra:** reservar stock → cobrar → confirmar; si un paso falla, se deshacen los anteriores. Con clave de idempotencia para que un reintento no cobre dos veces.

4. Rendimiento web en móviles

- **Carga diferida:** cada pantalla descarga solo su código. El bundle inicial bajó de 691 kB a 351 kB.
- **Fuentes livianas:** la fuente de íconos pasó de 170 KB a 6 KB, y las fuentes se sirven desde el propio servidor.
- **Carga visible al instante:** pantalla de carga y esqueletos; el primer contenido aparece en 0,7 s en lugar de 4–5 s.
- **Menos datos:** paginación en el servidor, búsqueda con espera (debounce), compresión gzip y caché de archivos estáticos.
- **Siguiente paso:** convertirla en PWA para que funcione con mala señal en zonas rurales.

5. Seguridad en APIs y cloud

- **OAuth vs JWT:** JWT es un formato de token; OAuth es un protocolo para dar permisos. Para una sola app basta JWT; con varias apps o terceros conviene OAuth/OIDC.
- **Fuerza bruta:** máximo 5 intentos de login por minuto, contraseñas con bcrypt y mensaje de error genérico. En producción: MFA y CAPTCHA.
- **MongoDB:** siempre con usuario y clave, sin exposición a internet, usuario de la app con permisos mínimos y validación de datos para evitar inyecciones.
- **Cloud:** HTTPS, servicios en red privada, secretos en un gestor (nunca en el código), contenedores sin root y revisión de dependencias en CI.